

Tugas 1 - Queue Sederhana Menggunakan List

```
queue = []

queue.append("Piti")
queue.append("Mutia")
queue.append("Sarah")
queue.append("Lagi singel")
queue.append("Hanif")
queue.append("El Kecepatan")

print("Isi Queue:", queue)
```

Isi Queue: ['Piti', 'Mutia', 'Sarah', 'Lagi singel', 'Hanif', 'El Kecepatan']

Tugas 2 - Operasi Enqueue dan Dequeue

```
queue = []

# Enqueue
queue.append("Piti")
queue.append("Mutia")
queue.append("Sarah")
queue.append("Lagi singel")
queue.append("Hanif")
queue.append("El Kecepatan")

print("Queue setelah Enqueue:", queue)

# Dequeue
keluar = queue.pop(0)

print("Data keluar:", keluar)
print("Queue setelah Dequeue:", queue)
```

Queue setelah Enqueue: ['Piti', 'Mutia', 'Sarah', 'Lagi singel', 'Hanif', 'El Kecepatan']
Data keluar: Piti
Queue setelah Dequeue: ['Mutia', 'Sarah', 'Lagi singel', 'Hanif', 'El Kecepatan']

Tugas 3 - Menampilkan Front dan Rear

```
queue = []

queue.append("Piti")
queue.append("Mutia")
queue.append("Sarah")
queue.append("Lagi singel")
queue.append("Hanif")
queue.append("El Kecepatan")

print("Isi Queue:", queue)

print("Front:", queue[0])
print("Rear:", queue[-1])
```

Isi Queue: ['Piti', 'Mutia', 'Sarah', 'Lagi singel', 'Hanif', 'El Kecepatan']
Front: Piti
Rear: El Kecepatan

Tugas 4 - Menu Queue Interaktif

```
queue = []

while True:
    print("\n=== MENU QUEUE ===")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Front")
    print("4. Rear")
    print("5. Tampilkan Queue")
    print("6. Keluar")
```

```
pilihan = input("Pilih menu: ")

if pilihan == "1":
    data = input("Masukkan data: ")
    queue.append(data)

elif pilihan == "2":
    if len(queue) == 0:
        print("Queue kosong.")
    else:
        print("Data keluar:", queue.pop(0))

elif pilihan == "3":
    if len(queue) == 0:
        print("Queue kosong.")
    else:
        print("Front:", queue[0])

elif pilihan == "4":
    if len(queue) == 0:
        print("Queue kosong.")
    else:
        print("Rear:", queue[-1])

elif pilihan == "5":
    print("Isi Queue:", queue)

elif pilihan == "6":
    break

else:
    print("Pilihan tidak valid.")
```

```
=== MENU QUEUE ===
1. Enqueue
2. Dequeue
3. Front
4. Rear
5. Tampilkan Queue
6. Keluar
Pilih menu: 1
Masukkan data: ga jelas

=== MENU QUEUE ===
1. Enqueue
2. Dequeue
3. Front
4. Rear
5. Tampilkan Queue
6. Keluar
Pilih menu: 2
Data keluar: Apa
```

1. Enqueue

2. Dequeue

3. Front

4. Rear

5. Tampilkan Queue

6. Keluar
- Pilih menu: 6

Tugas 5 - Simulasi Antrian Kasir Minimarket

```
antrian = []

antrian.append("Pelanggan 1")
antrian.append("Pelanggan 2")
antrian.append("Pelanggan 3")
antrian.append("Pelanggan 4")

print("Antrian Awal:", antrian)

layani = antrian.pop(0)

print("Dilayani:", layani)
print("Sisa Antrian:", antrian)

Antrian Awal: ['Pelanggan 1', 'Pelanggan 2', 'Pelanggan 3', 'Pelanggan 4']
Dilayani: Pelanggan 1
Sisa Antrian: ['Pelanggan 2', 'Pelanggan 3', 'Pelanggan 4']
```

Tugas 6 - Queue Menggunakan deque

```
from collections import deque

queue = deque()

queue.append("Piti")
queue.append("Mutia")
queue.append("Sarah")
queue.append("Lagi singel")
queue.append("Hanif")
queue.append("El Kecepatan")

print("Queue:", queue)

keluar = queue.popleft()

print("Data keluar:", keluar)
print("Queue sekarang:", queue)

Queue: deque(['Piti', 'Mutia', 'Sarah', 'Lagi singel', 'Hanif', 'El Kecepatan'])
Data keluar: Piti
Queue sekarang: deque(['Mutia', 'Sarah', 'Lagi singel', 'Hanif', 'El Kecepatan'])
```

Tugas 7 - Perbandingan Queue List dan Deque

```
from collections import deque

# Queue menggunakan List
queue_list = []

queue_list.append("Dudung")
queue_list.append("Diding")
queue_list.append("Dadang")

print("Queue List:")
print(queue_list)

keluar_list = queue_list.pop(0)

print("Data keluar:", keluar_list)
print("Sisa Queue:", queue_list)

print("\n=====")

# Queue menggunakan Deque
queue_deque = deque()
```

```
queue_deque.append("Dudung")
queue_deque.append("Diding")
queue_deque.append("Dadang")

print("Queue Deque:")
print(queue_deque)

keluar_deque = queue_deque.popleft()

print("Data keluar:", keluar_deque)
print("Sisa Queue:", queue_deque)
```

```
Queue List:
['Dudung', 'Diding', 'Dadang']
Data keluar: Dudung
Sisa Queue: ['Diding', 'Dadang']

=====

Queue Deque:
deque(['Dudung', 'Diding', 'Dadang'])
Data keluar: Dudung
Sisa Queue: deque(['Diding', 'Dadang'])
```

Penjelasan No.7

Queue dapat diimplementasikan menggunakan struktur data List maupun Deque. Pada program ini dilakukan percobaan menggunakan kedua metode tersebut dengan operasi yang sama, yaitu menambahkan data (Enqueue) dan menghapus data (Dequeue). Saat menggunakan List, proses penambahan data dilakukan dengan fungsi append(), sedangkan penghapusan data dilakukan dengan pop(0). Fungsi pop(0) akan menghapus elemen pertama dalam list, tetapi seluruh elemen setelahnya harus bergeser satu posisi ke depan. Oleh karena itu, proses ini membutuhkan waktu yang lebih lama ketika jumlah data semakin banyak.

Berbeda dengan List, Deque yang berasal dari modul collections menyediakan fungsi append() untuk menambah data dan popleft() untuk menghapus data paling depan. Fungsi popleft() dapat langsung menghapus elemen pertama tanpa harus menggeser elemen lainnya sehingga prosesnya lebih cepat dan efisien. Dari hasil perbandingan tersebut dapat disimpulkan bahwa Deque lebih cocok digunakan untuk implementasi Queue karena memiliki performa yang lebih baik, terutama ketika jumlah data dalam antrian sangat banyak.

Tugas 8 - Simulasi Sistem Antrian Rumah Sakit

```
pasien = deque()

while True:
    print("\n=== ANTRIAN RUMAH SAKIT ===")
    print("1. Tambah Pasien")
    print("2. Panggil Pasien")
    print("3. Lihat Antrian")
    print("4. Keluar")

    pilihan = input("Pilih menu: ")

    if pilihan == "1":
        nama = input("Nama pasien: ")
        pasien.append(nama)

    elif pilihan == "2":
        if len(pasien) == 0:
            print("Tidak ada pasien.")
        else:
            print("Pasien dipanggil:", pasien.popleft())

    elif pilihan == "3":
        print("Antrian:", list(pasien))

    elif pilihan == "4":
        break

    else:
        print("Pilihan salah.")
```

```
=== ANTRIAN RUMAH SAKIT ===
1. Tambah Pasien
2. Panggil Pasien
3. Lihat Antrian
4. Keluar
```

```
Pilih menu: 1
Nama pasien: Jubed

=== ANTRIAN RUMAH SAKIT ===
1. Tambah Pasien
2. Panggil Pasien
3. Lihat Antrian
4. Keluar
Pilih menu: 1
Nama pasien: siska

=== ANTRIAN RUMAH SAKIT ===
1. Tambah Pasien
2. Panggil Pasien
3. Lihat Antrian
4. Keluar
Pilih menu: 1
Nama pasien: eneng

=== ANTRIAN RUMAH SAKIT ===
1. Tambah Pasien
2. Panggil Pasien
3. Lihat Antrian
4. Keluar
Pilih menu: 1
Nama pasien: tini

=== ANTRIAN RUMAH SAKIT ===
1. Tambah Pasien
2. Panggil Pasien
3. Lihat Antrian
4. Keluar
Pilih menu: 1
Nama pasien: dadang

=== ANTRIAN RUMAH SAKIT ===
1. Tambah Pasien
2. Panggil Pasien
3. Lihat Antrian
4. Keluar
Pilih menu: 1
Nama pasien: dudung

=== ANTRIAN RUMAH SAKIT ===
1. Tambah Pasien
2. Panggil Pasien
3. Lihat Antrian
4. Keluar
Pilih menu: 1
Nama pasien: enok
```

Tugas 9 - Simulasi Printer Queue

```
printer = deque()

printer.append("Dokumen_A.pdf")
printer.append("Dokumen_B.pdf")
printer.append("Dokumen_C.pdf")

print("Antrian Cetak:", list(printer))

while printer:
    print("Mencetak:", printer.popleft())

print("Semua dokumen selesai dicetak.")

Antrian Cetak: ['Dokumen_A.pdf', 'Dokumen_B.pdf', 'Dokumen_C.pdf']
Mencetak: Dokumen_A.pdf
Mencetak: Dokumen_B.pdf
Mencetak: Dokumen_C.pdf
Semua dokumen selesai dicetak.
```

Tugas 10 - Simulasi Call Center Koperasi

```
telepon = deque()

telepon.append("Anggota 1")
telepon.append("Anggota 2")
telepon.append("Anggota 3")
```

```
print("Antrian Telepon:", list(telepon))

while telepon:
    print("Melayani:", telepon.popleft())

print("Semua panggilan selesai.")
```

Antrian Telepon: ['Anggota 1', 'Anggota 2', 'Anggota 3']
Melayani: Anggota 1
Melayani: Anggota 2
Melayani: Anggota 3
Semua panggilan selesai.

Tugas 11 - Sistem Playlist Musik Menggunakan Deque

```
\\playlist = deque()

playlist.append("Lagu A")
playlist.append("Lagu B")
playlist.append("Lagu C")

print("Playlist Awal:", list(playlist))

# Lagu berikutnya
playlist.rotate(-1)

print("Lagu Sekarang:", playlist[0])

# Kembali ke lagu sebelumnya
playlist.rotate(1)

print("Lagu Sebelumnya:", playlist[0])
```

Playlist Awal: ['Lagu A', 'Lagu B', 'Lagu C']
Lagu Sekarang: Lagu B
Lagu Sebelumnya: Lagu A

Tugas 12 – Analisis Kompleksitas Operasi pada Program Tugas 8–11

Pada simulasi antrian rumah sakit, operasi yang paling sering digunakan adalah menambahkan pasien ke dalam antrian menggunakan `append()` dan memanggil pasien menggunakan `popleft()`. Kedua operasi tersebut memiliki kompleksitas waktu $O(1)$, yang berarti waktu eksekusinya tetap dan tidak dipengaruhi oleh jumlah pasien yang ada dalam antrian. Namun, ketika seluruh isi antrian ditampilkan, program harus membaca setiap data pasien satu per satu sehingga kompleksitasnya menjadi $O(n)$, dengan n adalah jumlah pasien dalam antrian.

Pada simulasi printer queue, setiap dokumen yang masuk ke dalam antrian ditambahkan menggunakan `append()` dan setiap dokumen yang dicetak diambil menggunakan `popleft()`. Kedua operasi ini juga memiliki kompleksitas $O(1)$. Akan tetapi, jika seluruh dokumen dalam antrian dicetak atau ditampilkan, program harus memproses semua data sehingga kompleksitasnya menjadi $O(n)$.

Pada simulasi call center koperasi, proses penambahan panggilan baru ke dalam antrian dilakukan menggunakan `append()`, sedangkan panggilan yang akan dilayani diambil menggunakan `popleft()`. Sama seperti simulasi sebelumnya, kedua operasi tersebut memiliki kompleksitas $O(1)$. Ketika seluruh antrian panggilan ditampilkan atau diproses satu per satu, kompleksitas waktunya menjadi $O(n)$ karena program harus mengakses semua data yang ada.

Pada sistem playlist musik yang menggunakan Deque, penambahan lagu dilakukan dengan `append()` yang memiliki kompleksitas $O(1)$. Perpindahan ke lagu berikutnya atau lagu sebelumnya dilakukan menggunakan fungsi `rotate()`, yang pada kasus perpindahan satu langkah memiliki kompleksitas $O(1)$. Ketika seluruh daftar lagu ditampilkan, program harus membaca seluruh isi playlist sehingga kompleksitasnya menjadi $O(n)$.

Berdasarkan analisis tersebut, dapat disimpulkan bahwa penggunaan Deque sangat efektif untuk implementasi Queue karena operasi utama seperti penambahan dan penghapusan data dapat dilakukan dengan kompleksitas $O(1)$. Hal ini membuat Deque lebih efisien dibandingkan List, terutama pada aplikasi yang melibatkan banyak proses antrian seperti rumah sakit, printer, call center, maupun sistem playlist musik.

